Python Functions

Why Do We Use Functions?

Functions are used to:

- Avoid repetition of code
- Make programs reusable
- Improve readability
- Break large problems into smaller parts
- Make debugging easier

Rule: If you write the same code more than once, use a function.

Level 1: Basics

1. Define a function `hello()` that prints "Hello World".
   Hint: Use `def` keyword and `print()`.
2. Call the function `hello()` three times.
   Hint: Function call uses parentheses.
3. Write a function `print_name(name)`.
   Hint: Parameter goes inside parentheses.
4. Create a function that prints your age.
   Hint: No arguments needed.
5. Difference between function definition and function call?
   Hint: One creates the function, the other executes it.

Level 2: Positional Arguments

6. Write `add(a, b)` that prints sum.
   Hint: Use `a + b` inside function.
7. Write `multiply(a, b)`.
   Hint: Replace `+` with `*`.

8. Create `full_name(first, last)`.
   Hint: Print both values together.

9. What happens if fewer arguments are passed?
   Hint: Python throws an error.

10. What if more arguments are passed?
    Hint: Python does not allow extra positional arguments.

Level 3: Keyword Arguments

11. Call `full_name()` using keywords.
    Hint: Use `key=value` format.

12. Create `student_info(name, grade, section)`.
    Hint: Call using keyword arguments.

13. Change argument order using keywords.
    Hint: Order does not matter in keyword arguments.

14. Why are keyword arguments safer?
    Hint: They prevent order-based mistakes.

Level 4: *args

15. Write a function that prints all numbers passed.
    Hint: `*args` collects values as a tuple.

16. Write `total(*numbers)`.
    Hint: Use a loop and accumulator variable.

17. Call function with different number of arguments.
    Hint: Try 2, 5, and 10 values.

18. What is the type of `args`?
    Hint: Use `type()` to check.

19. Can loops be used inside `*args`?
    Hint: Yes, iterate like a tuple.

Level 5: **kwargs

20. Write `display_info(**kwargs)`.
    Hint: Use `kwargs.items()`.

21. Pass only name and age.
    Hint: Keyword arguments are optional.

22. Pass multiple details.
    Hint: Add more key=value pairs.

23. What is the type of `kwargs`?
    Hint: It behaves like a dictionary.

24. Why is `**kwargs` flexible?
    Hint: Accepts unlimited named arguments.

Level 6: Return Statement

25. Write `square(num)` that returns square.
    Hint: Use `return num * num`.

26. Store returned value in a variable.
    Hint: Assign function call to a variable.

27. Write `add(a, b)` using return.
    Hint: Avoid `print()`.

28. Difference between print and return?
    Hint: One displays, one sends value back.

29. Can a function return multiple values?
    Hint: Use tuple return.

Level 7: Thinking Questions

30. When to use `*args`?
    Hint: Unknown number of positional arguments.

31. When to use `**kwargs`?
    Hint: Unknown named arguments.

32. Why is return important?
    Hint: Needed for calculations and reuse.

33. Why does `sum((3,5))` work?
    Hint: Sum expects iterable.

34. Convert print-based function to return-based.
    Hint: Replace print with return.

Optional Challenges

35. Create calculator using `*args`.
    Hint: Loop through numbers.

36. Create student profile using `**kwargs`.
    Hint: Print keys and values.

37. Return total and average.
    Hint: Use tuple return.

38. Even or odd checker.
    Hint: Use modulus operator.

39. Find maximum from `*args`.
    Hint: Compare values in loop.

Example Code

```
def add(*numbers):
    result = 0
    for num in numbers:
        result += num
    return result

print(add(2, 3, 40, 577))
```

Menu Driven Programs — Modular Programming Using Functions

Important Instruction

In all questions below:

- The given program is **working but not modular**
- Your task is to **convert it into a modular program**
- Use **functions** to make the code:
  - Clean
  - Readable
  - Reusable
  - Easy to understand
- Avoid code duplication
- Prefer `return` instead of `print` inside functions

Goal: Main loop should look simple and readable.

—

Question 1: Unit Converter (Reference Example)

You have already seen and practiced this example in class.

```
MENU = """
########################
    1. km to m
    2. m to cm
    3. cm to mm
    4. Exit
########################
"""

def show_answer(ans, units):
    print("############################")
    print("")
    print(f"Ans: {ans} {units}")
    print("")
    print("############################")

def kilometer_to_meter():
    print("----Kilometer to meter converter------")
    km_value = float(input("Kilometer ? >>>  "))
    m_value = km_value * 1000
    return m_value

while True:
    print(MENU)
    user_choice = input("choice: ")

    if user_choice == "1":
        meter_ans = kilometer_to_meter()
        show_answer(ans=meter_ans, units="meters")

    elif user_choice == "2":
        print("----Meter to centimeter converter------")
        m_value = float(input("Meter ? >>>  "))
        cm_value = m_value * 100
        show_answer(ans=cm_value, units="centimeters")

    elif user_choice == "3":
        print("----Centimeter to millimeter converter------")
        cm_value = float(input("Centimeter ? >>>  "))
        mm_value = cm_value * 10
        show_answer(ans=mm_value, units="millimeters")

    elif user_choice == "4":
        break
    else:
        print("Please, enter correct choice")
```

Task:

- Convert menu printing into `show_menu()`
- Create functions for remaining conversions

- Keep main loop clean

—

Question 2: Menu Driven Calculator

Given Program (Non-Modular)

```
MENU = """
========================
        Calculator
    1. Add
    2. Subtract
    3. Multiply
    4. Divide
    5. Exit
========================
"""

while True:
    print(MENU)
    choice = input("Choice >>> ")

    if choice == "1":
        a = float(input("First number: "))
        b = float(input("Second number: "))
        print("Result:", a + b)

    elif choice == "2":
        a = float(input("First number: "))
        b = float(input("Second number: "))
        print("Result:", a - b)

    elif choice == "3":
        a = float(input("First number: "))
        b = float(input("Second number: "))
        print("Result:", a * b)

    elif choice == "4":
        a = float(input("First number: "))
        b = float(input("Second number: "))
        print("Result:", a / b)

    elif choice == "5":
        break
    else:
        print("Invalid choice")
```

Your Task

Convert the above program into a **modular version** by creating:

- `show_menu()`
- `get_numbers()`
- `add(a, b)`
- `subtract(a, b)`
- `multiply(a, b)`
- `divide(a, b)`
- `show_result(result)`

—

Question 3: Student Result System

Given Program (Non-Modular)

```
MENU = """
============================
    Student Result System
    1. Calculate Total
    2. Calculate Percentage
    3. Check Pass/Fail
    4. Exit
============================
"""

while True:
    print(MENU)
    choice = input("Choice >>> ")

    if choice == "1":
        m1 = int(input("Math: "))
        m2 = int(input("Science: "))
        m3 = int(input("English: "))
        print("Total:", m1 + m2 + m3)

    elif choice == "2":
        m1 = int(input("Math: "))
        m2 = int(input("Science: "))
        m3 = int(input("English: "))
        print("Percentage:", (m1 + m2 + m3) / 3)

    elif choice == "3":
        m1 = int(input("Math: "))
```

```
        m2 = int(input("Science: "))
        m3 = int(input("English: "))
        if m1 + m2 + m3 >= 120:
            print("PASS")
        else:
            print("FAIL")

    elif choice == "4":
        break
    else:
        print("Invalid choice")
```

Your Task

Refactor into a modular program using:

- `show_menu()`
- `get_marks()`
- `calculate_total(marks)`
- `calculate_percentage(marks)`
- `check_pass_fail(total)`
- `show_result(label, value)`

—

Question 4: Bank System

Given Program (Non-Modular)

```
balance = 5000

MENU = """
====================
    Bank System
    1. Check Balance
    2. Deposit
    3. Withdraw
    4. Exit
====================
"""

while True:
    print(MENU)
    choice = input("Choice >>> ")

    if choice == "1":
```

```
        print("Balance:", balance)

    elif choice == "2":
        amount = float(input("Deposit amount: "))
        balance += amount

    elif choice == "3":
        amount = float(input("Withdraw amount: "))
        if amount <= balance:
            balance -= amount
        else:
            print("Insufficient balance")

    elif choice == "4":
        break
```

Your Task

Convert this into a modular program using:

- `show_menu()`
- `check_balance(balance)`
- `deposit(balance)`
- `withdraw(balance)`

All functions must **return updated balance**.

Question 5: Temperature Converter

Given Program (Non-Modular)

```
MENU = """
==========================
    Temperature Converter
    1. Celsius to Fahrenheit
    2. Fahrenheit to Celsius
    3. Exit
==========================
"""

while True:
    print(MENU)
    choice = input("Choice >>> ")

    if choice == "1":
        c = float(input("Celsius: "))
```

```
        print((c * 9/5) + 32)

    elif choice == "2":
        f = float(input("Fahrenheit: "))
        print((f - 32) * 5/9)

    elif choice == "3":
        break
```

Your Task

Make this program fully modular by creating:

- `show_menu()`
- `celsius_to_fahrenheit(c)`
- `fahrenheit_to_celsius(f)`
- `show_answer(value, unit)`

—

Final Note

- Do not write everything inside `while True`
- Each function should do only **one job**
- Clean code is more important than short code
- This style is used in **real software projects**